

飞虹 Code 终端 AI 编程智能体软件 用户使用手册

V1.0.0

著作权人：吴赐虹 | 开发完成日期：2026年8月23日 | 首次发表日期：2026年8月23日

第一章 软件概述

1.1 产品简介

飞虹 Code（英文名称 fhcode）是一款面向终端开发者的 AI 编程智能体软件，由吴赐虹独立开发完成。软件对标国际主流的 Muse Code 和 Cursor CLI 产品，旨在通过大语言模型驱动的自主代理架构，实现软件开发全流程的自动化和智能化。软件支持多模型路由、全自动软件工程代理、企业级安全审计、代码评审与重构、技能系统与自进化学习等核心能力，同时支持完全离线的私有化部署模式。

软件采用 TypeScript 和 Node.js 技术栈开发，

具备良好的跨平台兼容性，支持 Windows 10/11（64位）、

macOS 11 及以上版本（含 Apple Silicon 芯片）以及主流 Linux 发行版（Ubuntu 20.04+、

CentOS 8+、Debian 10+ 等）。软件通过命令行界面（CLI）和 Web 控制台双入口提供服务，

开发者既可以在熟悉的终端环境中进行交互式编程对话，也可以通过浏览器访问 Web 控制台进行任务管理、

状态监控和日志审计。

软件内置灵活的模型路由层，支持接入多种主流大语言模型服务，包括但不限于 DeepSeek、通义千问、Ollama 本地模型以及任意兼容 OpenAI API 协议的第三方模型服务。

用户可根据自身需求和预算，配置一个或多个模型提供商，软件会根据预设的策略（成本优先、

能力优先、延迟优先)自动选择最优模型进行调用。当某个模型服务出现故障或限流时,路由层会自动切换到其他可用模型,保证服务的连续性和稳定性。

飞虹 Code 的核心理念是让 AI 成为真正的编程伙伴,而非简单的代码补全工具。

与传统的代码补全插件不同,飞虹 Code 通过自主代理架构,能够理解复杂的软件开发目标,

自动将大目标拆解为可执行的子任务序列,调用文件读写、代码搜索、Shell 执行、构建验证、测试运行等多种工具,完成从需求分析到代码实现、从测试验证到文档生成的完整开发闭环。

软件还具备强大的自我修复能力,当构建失败或测试不通过时,代理能够自动分析错误原因并迭代修复,

直至任务成功完成。

1.2 技术架构

飞虹 Code 采用分层模块化的软件架构设计,自下而上分为运行时层、工具层、模型层、代理层、技能层和表现层六个层次。各层之间通过清晰的接口进行通信,具备良好的可扩展性和可维护性。运行时层提供软件运行所需的基础能力,包括 Git 工作区管理、会话持久化存储、事件日志系统、钩子机制、工作树管理等;工具层封装了代理可调用的各种工具,包括文件操作工具、代码搜索工具、Shell 执行工具、构建验证工具、Web 浏览工具、MCP 客户端工具等;模型层实现了多模型供应商适配、自动择优路由、异常轮换与退避重试等能力。

代理层是软件的核心调度引擎,包含编排器、规划器、代码生成器、质量门控、自我修复、上下文压缩、经验管理、子代理等核心组件。编排器采用思考-行动-观察循环模式驱动代理执行任务,

接收用户目标后调用规划器进行任务拆解,然后依次调用模型生成工具调用、

执行工具并观察结果,循环迭代直至任务完成或达到最大轮次限制。自我修复模块在构建或测试失败时自动触发,

对错误进行分类并生成反思提示，引导模型修正方案。技能层提供可扩展的技能框架，允许用户为代理添加特定领域的专业技能。表现层包括 CLI 交互界面和 Web 控制台。

1.3 应用场景

飞虹 Code 适用于多种软件开发和运维场景。在日常编码场景中，开发者可以通过自然语言描述开发需求，代理自动生成代码文件、修改现有代码、执行搜索替换、添加依赖配置，大幅提升编码效率。在代码评审场景中，软件内置结构化代码评审引擎，能够扫描代码库中的潜在问题，包括安全漏洞、性能瓶颈、代码规范违规、复杂度过高、可维护性差等，按严重程度分为四个等级输出评审报告和修复建议。

在自动化测试场景中，软件能够根据代码变更自动生成单元测试用例，执行测试并分析覆盖率，

当测试失败时自动修复测试代码或被测代码。在遗留系统重构场景中，代理能够理解现有代码结构，

制定渐进式重构计划，逐步执行代码迁移、架构调整、依赖升级，同时保证构建和测试持续通过。

在 DevOps 场景中，软件支持通过 Webhook 触发自动化任务，与 CI/CD 流水线深度集成，实现代码提交后的自动评审、自动测试、自动部署全流程。在企业级应用场景中，飞虹 Code 提供多租户隔离、基于角色的访问控制、操作审计日志、资源配额管理等企业级能力。

1.4 系统要求

飞虹 Code 的运行环境要求如下。操作系统方面，支持 Windows 10 版本 21H2 及以上（64 位）、

Windows 11 全部版本、macOS 11 Big Sur 及以上版本（支持 Intel 和 Apple Silicon 芯片）、

Linux 内核版本 4.15 及以上的主流发行版。运行时环境方面，

需要 Node.js 18.0.0 及以上版本，推荐使用 Node.js 20 LTS 或 22 LTS 版本以获得最佳性能和稳定性。

硬件资源方面，最低配置要求 2GB 系统内存、500MB 可用磁盘空间、

x86_64 或 ARM64 架构处理器。推荐配置为 4GB 以上系统内存、2GB 可用磁盘空间、多核处理器。对于使用 Ollama 本地模型的用户，推荐至少 8GB 显存的 GPU。

第二章 安装部署

2.1 npm 安装

飞虹 Code 已发布到 npm 公共软件仓库，用户可以通过 npm 命令进行全局安装，这是最推荐的安装方式。在安装前，请确保系统已安装 Node.js 18.0.0 及以上版本，可以在终端中执行 `node --version` 命令验证 Node.js 版本。在 Windows 系统中，打开命令提示符或 PowerShell 终端，执行 `npm install -g feihong-code` 命令进行全局安装。安装完成后执行 `fhcode --version` 命令验证安装是否成功。如果遇到权限错误，可以尝试以管理员身份运行终端后重新执行安装命令。

在 macOS 和 Linux 系统中，打开终端应用，执行 `sudo npm install -g feihong-code` 命令进行全局安装。

如果遇到 EACCES 权限错误，除了使用 `sudo` 外，还可以修改 npm 全局安装目录到用户主目录下，

具体方法为执行 `npm config set prefix ~/.npm-global` 设置新的全局目录，然后将 `~/.npm-global/bin` 添加到 PATH 环境变量中。

软件的升级和卸载同样通过 npm 命令完成：升级执行 `npm update -g feihong-code`，查看版本执行 `npm list -g feihong-code`，卸载执行 `npm uninstall -g feihong-code`。

安装完成后，首次运行 `fhcode` 命令会自动在用户主目录下创建 `.feihong-code` 配置目录。

2.2 Docker 部署

飞虹 Code 提供官方 Docker 镜像，支持容器化部署，特别适合企业级私有化部署和 CI/CD 环境。

使用 Docker 部署可以避免环境依赖问题，保证运行环境的一致性，

同时便于快速扩展和迁移。在部署前，请确保系统已安装 Docker 20.10 及以上版本和 Docker Compose v2。

使用 `docker run` 命令快速启动单容器实例：`docker run -d --name fhcode -p 8080:8080 -e FH_PROVIDERS='[...]' -e FH_WEB_TOKEN=your-token feihong-code:latest`。

启动后通过浏览器访问 `http://localhost:8080` 即可打开 Web 控制台。

推荐使用 `docker-compose.yml` 进行部署，便于管理配置和数据持久化。

创建 `docker-compose.yml` 文件，配置服务名称、

镜像版本、端口映射、环境变量、数据卷挂载等选项。环境变量中配置 `FH_PROVIDERS`（模型提供商 JSON 数组）、

`FH_MODEL_NAME`（默认模型名称）、`FH_WEB_PORT`（Web 控制台端口）、

`FH_WEB_TOKEN`（Web 控制台访问令牌）、`FH_ENTERPRISE`（是否启用企业级功能）等。

`volumes` 中挂载配置目录和数据目录到宿主机，实现数据持久化。

配置完成后执行 `docker compose up -d` 启动服务，

`docker compose logs -f` 查看实时日志，`docker compose down` 停止并移除服务。

企业级部署建议通过 Nginx 或 Traefik 反向代理提供 HTTPS 访问，

配置 SSL 证书。

2.3 源码编译安装

对于希望从源代码编译安装的用户，可以从 GitHub 仓库克隆源代码后进行本地构建。

这种方式适合开发者参与项目贡献、自定义修改或使用最新开发版本。在编译前，

请确保系统已安装 Git、Node.js 18+ 和 npm。克隆源代码：

执行 `git clone https://github.com/wch887292/feihong-code.git` 命令克隆仓库到本地，然后 `cd feihong-code` 进入项目目录。安装依赖：执行 `npm install` 命令安装项目依赖。

编译构建：执行 `npm run build` 命令进行 TypeScript 编译和 Web 静态资源复制。

构建脚本会先执行 `tsc` 命令将 TypeScript 源码编译为 JavaScript，

输出到 `dist` 目录，然后执行 `node scripts/copy-web.cjs` 脚本将 Web 控制台的静态资源复制到 `dist/web/public` 目录。

本地运行：执行 `npm start` 或 `node dist/cli/index.js` 命令启动软件。

全局链接：执行 `npm link` 命令将本地构建的软件链接到全局 `npm` 目录，

之后就可以在任意目录使用 `fhcode` 命令，适合开发调试。运行测试：

执行 `npm test` 命令运行单元测试套件，验证代码功能正确性；

执行 `npm run verify` 命令运行全量验证套件，确保各模块功能正常。

2.4 模型配置

模型配置是飞虹 Code 正常运行的关键步骤，通过 `FH_PROVIDERS` 环境变量配置一个或多个模型提供商。

`FH_PROVIDERS` 为 JSON 数组格式，每个数组元素描述一个模型提供商的完整配置信息，

包括 `id`（唯一标识）、`type`（类型：`openai-compatible` 或 `ollama`）、

`baseUrl`（API 基础地址）、`apiKey`（API 访问密钥）、

`tags`（能力标签数组）、`costPer1k`（每千 token 成本）等字段。

配置示例中包含三个提供商：`deepseek`（OpenAI 兼容接口，

地址 `https://api.deepseek.com/v1`，标签 `code-gen` 和 `cheap`）、

`qwen`（通义千问兼容接口，地址 `https://dashscope.aliyuncs.com/compatible-mode/v1`，

标签 `code-gen` 和 `long-context`）、`ollama`（本地模型，

地址 `http://localhost:11434`，标签 `code-gen` 和 `local`，

成本为 0）。

配置完成后，软件会根据默认的成本优先策略自动选择最优模型。用户可以通过 `--strategy` 命令行参数临时切换策略（`cost/capability/latency`），

或在 Web 控制台的模型管理页面手动指定当前任务使用的模型。对于未配置 FH_PROVIDERS 的情况，

软件自动进入离线模式，使用内置的脚本化 Mock 模型提供商驱动代理执行任务。

离线模式下所有功能均可正常体验，仅模型生成的内容为脚本化示例。

2.5 环境验证

安装和配置完成后，建议进行环境验证以确保软件正常运行。版本验证：

执行 `fhcode --version` 命令，确认输出版本号与预期一致。

离线模式验证：在未配置 FH_PROVIDERS 的情况下，执行 `fhcode run` 创建一个 `test.txt` 文件写入 `hello` 命令，

观察代理是否能够完成文件创建操作。在线模式验证：配置 FH_PROVIDERS 后，

执行 `fhcode run` 用一句话介绍你自己 命令，观察代理是否能够调用真实模型生成回复。

Web 控制台验证：执行 `fhcode web` 命令启动 Web 控制台，

在浏览器中访问 `http://localhost:8080`，创建一个测试任务观察执行情况。

第三章 快速上手

3.1 首次启动

安装完成后，在终端中执行 `fhcode` 命令即可启动飞虹 Code 的交互式对话界面。

首次启动时，软件会进行初始化操作，包括在用户主目录下创建 `.feihong-code` 配置目录、生成默认配置文件、初始化会话存储、加载内置技能等。初始化完成后，终端会显示欢迎信息，包括软件名称、版本号、当前模式（在线/离线）、配置的模型数量等信息，并提示用户输入开发需求。如果未配置 FH_PROVIDERS 环境变量，软件会自动进入离线模式，并在欢迎信息中提示当前为离线模式。

启动交互式对话后，用户可以直接输入自然语言描述开发需求，例如创建一个 Python 的 FastAPI 服务器。

代理接收到目标后，会自动进行任务规划，将大目标拆解为可执行的子任务序列，然后依次执行。执行过程中，终端会实时显示当前执行步骤、调用的工具名称、工具输入参数、工具执行结果、代理思考过程等信息。任务完成后，代理会输出总结报告，包括完成的子任务列表、生成或修改的文件清单、构建和测试验证结果、耗时统计等信息。用户可以继续输入新的需求进行迭代开发，软件支持上下文记忆。

3.2 交互式对话操作

在交互式对话界面中，用户可以通过自然语言与代理进行多轮对话协作。

基本操作包括：输入需求后按回车键发送，代理开始执行；执行过程中按 `Ctrl+C` 可以中断当前执行；

使用上下方向键浏览历史输入记录；使用 `Tab` 键自动补全文件名和命令。

对话过程中支持多种斜杠命令，用于快速切换模式或触发特定功能：`/plan` 切换到规划模式，

代理仅输出任务规划方案而不执行；`/grill` 对当前工作区的代码进行深度拷问式审查；

`/review 路径` 对指定目录或文件进行结构化代码评审；`/goal` 启动目标驱动模式；

`/clear` 清空当前对话上下文；`/exit` 或 `/quit` 退出对话界面；

`/help` 查看所有可用的斜杠命令。

多轮对话上下文管理：软件自动维护对话上下文，包括历史消息、已执行的工具调用、文件变更记录等。代理在后续对话中能够引用之前的操作结果，实现连贯的迭代开发。

当对话过长导致上下文超出模型限制时，软件自动触发上下文压缩，将历史对话摘要为紧凑形式，

保留关键信息的同时减少 `token` 占用。输出格式说明：代理的输出分为思考区域、

工具调用区域、工具结果区域、最终回复区域。工具调用区域使用不同的颜色区分不同类型的工具，

便于用户快速识别。

3.3 命令行参数详解

飞虹 Code 提供丰富的命令行参数，支持灵活配置运行行为。全局参数（适用于所有子命令）包括：

`--version` 或 `-v` 显示版本号并退出；`--help` 或 `-h` 显示帮助信息并退出；

`--offline` 强制使用离线模式；`--strategy` 指定模型选择策略（`cost/capability/latency`）；

`--model` 指定使用的模型名称；`--max-cycles` 指定最大执行轮次（默认 50）；

`--config` 指定配置文件路径；`--log-level` 指定日志级别（`debug/info/warn/error`）。

`fhcode chat` 子命令参数包括 `--goal` 启动时直接执行指定目标、

`--context-file` 注入上下文文件、`--no-welcome` 不显示欢迎信息。

`fhcode run` 子命令参数包括 `goal`（位置参数，必填）要执行的任务目标、

`--json` 输出 JSON 格式结果、`--output` 将结果输出到指定文件、

`--timeout` 任务超时时间。`fhcode review` 子命令参数包括 `path`（位置参数，

可选）要评审的目录或文件路径、`--json` 输出 JSON 格式、

`--threshold` 阻断阈值、`--rules` 启用规则列表、

`--exclude` 排除文件模式。`fhcode web` 子命令参数包括 `--port` 指定监听端口、

`--host` 指定监听地址、`--token` 指定访问令牌、`--no-browser` 启动后不自动打开浏览器。

3.4 离线模式使用指南

离线模式是飞虹 Code 的特色功能，当未配置 `FH_PROVIDERS` 环境变量或使用 `--offline` 参数时，

软件自动进入离线模式。在离线模式下，代理使用内置的脚本化 Mock 模型提供商驱动执行流程，

Mock 模型根据预设的脚本生成工具调用和回复，完整模拟真实代理的思考-行动-观察循环。

离线模式的核心价值在于：无需 API 密钥即可体验完整功能，适合演示、

培训、测试和 CI/CD 环境；所有操作在本地完成，无数据外泄风险，

适合安全敏感场景；执行速度快，不依赖网络延迟，适合快速验证工作流。

离线模式下，代理能够执行真实的文件读写、Shell 命令、代码搜索、构建验证等操作，生成的文件和执行结果与在线模式一致，仅模型生成的内容为脚本化示例。

离线模式支持自定义 Mock 脚本，高级用户可以通过编写自定义的 Mock 步骤序列来模拟特定的代理行为。

从离线模式切换到在线模式：设置 `FH_PROVIDERS` 环境变量并配置至少一个有效模型提供商，

然后重启软件或启动新会话即可使用真实大模型。用户可以根据实际需求在两种模式间切换，

开发调试阶段使用离线模式快速验证流程，正式开发阶段使用在线模式获得高质量生成结果。

第四章 核心功能详解

4.1 多模型路由

多模型路由是飞虹 Code 的核心能力之一，允许用户同时配置多个模型提供商，软件根据预设策略自动选择最优模型进行调用。支持的模型选择策略包括三种：
`cost`（成本优先）选择每千 token 成本最低的可用模型，适合预算敏感场景；
`capability`（能力优先）根据任务类型匹配具备相应能力标签的模型，

例如代码生成任务优先选择带有 `code-gen` 标签的模型；`latency`（延迟优先）选择历史平均响应时间最短的模型，

适合交互体验敏感的场景。路由层维护每个模型的实时性能统计数据，包括总调用次数、成功调用次数、失败调用次数、累计成本、平均延迟、最后使用时间、成功率等关键指标。统计数据以 JSONL 格式持久化到用户主目录下的 `model-stats.jsonl` 文件。

异常处理和自动轮换机制是路由层的重要组成部分。当上游模型返回错误时，路由层根据错误类型采取不同的处理策略：对于 400/401/403/404 等鉴权或模型类错误，立即轮换到下一个可用模型，不重试同一模型；对于 429/500/502/503/504 等限流或服务端瞬时错误，

轮换到下一个可用模型并进行指数退避后重试整轮，直至任务完成或达到最大重试次数；对于网络连接错误、超时、DNS 解析失败等网络类错误，按可重试错误处理，

同样触发模型轮换和退避重试。模型路由层支持运行时动态添加和移除模型提供商，无需重启软件，用户可以通过 Web 控制台的模型管理页面进行管理。

4.2 SWE Agent 全自动软件工程代理

SWE Agent (Software Engineering Agent) 是飞虹 Code 的高级代理模式，

专为复杂软件工程任务设计。与普通对话模式不同，SWE Agent 采用规划-执行-验证-修复的完整闭环架构，

能够自主处理从需求理解到代码实现、测试验证的全流程。SWE Agent 特别适合处理涉及多文件修改、

架构调整、依赖升级、Bug 修复、功能开发等复杂开发任务，能够显著减少人工干预，提升开发效率和代码质量。SWE Agent 的执行流程分为四个阶段：

规划阶段由 SWE 规划器接收用户目标进行深度分析，生成结构化的执行计划，

包括任务拆解、文件变更预估、风险评估、验证策略等；执行阶段代理按照规划的子任务序列逐步执行，

调用文件操作、代码搜索、Shell 执行、构建验证等工具。

验证阶段由 SWE 验证器在执行过程中持续监控构建状态和测试结果，

每当代理完成可能影响构建或测试的操作后，验证器自动触发构建检查和测试运行，

确认代码变更没有引入新的错误。自我修复阶段当工具执行失败或验证不通过时自动触发，

对错误进行深度分析和迭代修复。自我修复模块首先对错误进行分类（文件不存在、

构建错误、命令未找到、参数无效、测试失败、依赖冲突、权限不足、未知错误等），

然后根据错误类型生成针对性的反思提示，注入到代理的上下文中，引导模型分析错误的根本原因并制定修复方案。

代理根据反思提示重新规划修复步骤，执行修复操作后再次验证，循环迭代直至问题解决或达到最大修复轮次。

4.3 代码评审引擎

飞虹 Code 内置结构化代码评审引擎，能够对代码库进行自动化深度审查，

帮助开发者发现潜在问题、提升代码质量。评审引擎基于预定义的规则集，

扫描代码中的各类问题，按严重程度分为四个等级：`critical`（严重）表示存在安全漏洞、数据丢失风险、严重性能问题或核心功能缺陷，必须立即修复；`high`（高）表示存在明显的代码质量问题、

安全隐患或性能瓶颈，建议尽快修复；`medium`（中）表示存在代码规范违规、可维护性问题或轻微性能问题；`low`（低）表示存在代码风格问题、注释缺失或优化建议。评审规则覆盖安全漏洞、性能瓶颈、代码规范、复杂度、可维护性、错误处理、资源泄漏等多个维度。

代码评审支持多种使用方式：命令行方式执行 `fhcode review 路径` 对指定目录或文件进行评审，

默认输出人类可读的文本报告，`--json` 参数可输出 JSON 格式的结构化评审结果，`--threshold` 参数可设置阻断阈值，当发现该级别及以上问题时返回非零退出码，可接入 CI/CD 流水线实现代码质量门禁；编辑器集成方式中 VSCode 扩展支持内联评审，评审结果映射为编辑器诊断，在问题代码下方显示波浪线，悬停可查看问题详情和修复建议，

CodeAction 提供一键快速修复，保存文件时可自动触发评审；

CI/CD 集成方式中评审命令可接入持续集成流水线。评审引擎支持高度可配置性，

用户可以通过配置文件启用或禁用特定规则，调整规则的严重程度，添加项目特定的自定义评审规则。

4.4 技能系统

技能系统是飞虹 Code 的可扩展能力框架，允许用户为代理添加特定领域的专业技能，使代理能够处理特定类型的任务。每个技能是一个独立的配置文件（YAML 或 Markdown 格式），

描述技能的元数据（名称、描述、版本、作者、标签）、触发条件（在什么情况下自动激活该技能）、

执行步骤（技能执行的具体流程，包括提示模板、工具调用、条件判断、循环等）、输入输出规范等。代理在执行任务时，根据目标描述和当前上下文自动匹配并加载相关技能，

按照技能定义的流程执行，从而获得特定领域的专业能力。软件内置多个基础技能：

`goal`（目标驱动）技能帮助代理将模糊的高层目标转化为具体可执行的行动计划；

plan（规划）技能提供结构化的任务拆解方法论；grill（拷问）技能模拟资深架构师对代码进行深度审查；

self-heal（自我修复）技能提供错误分类体系和修复策略模板。

技能市场是飞虹 Code 的社区生态组成部分，用户可以从技能市场下载社区贡献的各类技能，

扩展代理的专业能力。技能市场涵盖前端开发、后端开发、DevOps、数据科学、安全审计、文档生成等多个领域。用户也可以将自己编写的技能发布到技能市场，

与社区分享。编写自定义技能：用户可以为特定项目或团队编写自定义技能文件，放置到技能目录（`~/feihong-code/skills/` 或项目目录下的 `.fhcode-skills/`）中，软件启动时自动加载。技能文件采用 YAML 格式，包含 name、description、triggers、steps、output 等字段。

技能支持变量插值、条件分支、循环迭代、子技能调用等高级特性，能够编写复杂的工作流。

自定义技能特别适合将团队的最佳实践、编码规范、项目特定流程固化为可复用的技能，提升团队整体开发效率和代码一致性。

4.5 自进化与经验学习

自进化是飞虹 Code 的高级特性，使代理能够在执行任务过程中自动积累经验，从成功和失败中学习，持续提升执行能力和效率。自进化系统的核心是经验管理模块，该模块记录每次任务执行的完整轨迹，包括任务目标、规划方案、工具调用序列、执行结果、验证状态、最终成败、耗时统计、成本统计等关键信息。经验数据以结构化格式持久化存储，

支持高效检索和分析。当代理遇到类似任务时，可以检索历史经验，借鉴成功的执行模式和工具调用序列，

避免重复之前的错误，从而提升首次成功率和执行效率。经验积累机制：

每次任务执行完成后，经验模块自动将执行轨迹归档为一条经验记录。经验记录包含任务的语义特征、

执行特征、结果特征。经验记录支持标签化管理，用户可以为经验记录添加自定义标签，便于分类和检索。

经验检索与应用：当代理接收新任务时，经验模块自动根据任务目标的语义特征检索历史经验，

找到最相似的成功经验和失败经验。成功经验作为正面参考，提示代理可以采用类似的工具调用序列和执行策略；

失败经验作为反面参考，提示代理避免类似的错误和陷阱。检索到的经验以结构化提示的形式注入到代理的上下文中，

代理可以参考经验制定更优的执行计划。经验检索支持多种相似度算法，

包括基于关键词的匹配、基于向量嵌入的语义相似度、基于工具调用序列的结构相似度等。

自进化闭环：飞虹 Code 的自进化系统形成完整的执行-记录-检索-应用-优化闭环。

代理执行任务时自动积累经验，后续任务参考历史经验提升表现，新的执行结果又反馈到经验库中，

持续丰富和优化经验数据。随着使用时间的增长，经验库不断积累，代理对特定项目和技术栈的理解不断加深，

执行效率和成功率持续提升。需要注意的是，自进化系统仅记录执行轨迹和元数据，

不记录敏感信息，经验数据默认存储在本地，用户可以自主控制数据的导出和共享。

第五章 Web 控制台

5.1 控制台概述与启动

飞虹 Code 提供基于浏览器的 Web 控制台，允许用户通过图形界面管理代理任务、监控执行状态、查看审计日志、配置系统参数。Web 控制台采用前后端分离架构，后端基于 Express.js 提供 RESTful API 和 WebSocket 实时推送服务，前端使用原生 HTML/CSS/JavaScript 实现，无需额外的构建工具，轻量高效，加载速度快。控制台支持响应式布局，适配桌面端和移动端浏览器，在手机和平板上也能正常使用。启动 Web 控制台的方式有多种：命令行方式执行 `fhcode web` 命令启动 Web 控制台服务，

默认监听 8080 端口，可通过 `--port` 参数或 `FH_WEB_PORT` 环境变量修改端口号，`--host` 参数指定监听地址，`--token` 参数指定访问令牌；

Docker 方式中 Web 控制台随容器自动启动；系统服务方式中在 Linux 系统中可以配置 `systemd` 服务，

设置为开机自启动。

访问控制台：在浏览器中输入 `http://服务器地址:端口号` 访问 Web 控制台。

如果配置了 `FH_WEB_TOKEN` 环境变量，首次访问会要求输入访问令牌，

输入正确后进入控制台主界面。建议在生产环境中配置 `FH_WEB_TOKEN` 启用身份验证，

防止未授权访问。企业版部署建议通过 Nginx 或 Traefik 反向代理提供 HTTPS 访问，

并配置 SSL 证书，保障数据传输安全。控制台主界面采用左侧导航栏加右侧内容区的经典布

局，

左侧导航栏包含任务管理、模型管理、审计日志、企业管理（企业版）、

系统配置、系统监控等功能模块入口，右侧内容区显示当前选中模块的详细内容。

控制台的核心技术特性包括：实时推送（通过 WebSocket 实现任务状态变更和执行输出的实时推送）、

响应式设计（适配不同屏幕尺寸）、国际化支持（界面文本支持中英文切换）、

数据可视化（任务统计、模型性能、系统资源等数据以图表形式直观展示）、

操作审计（所有用户操作均记录审计日志）。

5.2 任务管理

任务管理是 Web 控制台的核心功能模块，用户可以在控制台中创建新任务、

查看任务列表、监控任务执行状态、查看任务详情和完整输出、取消正在执行的任务、

删除历史任务记录。任务采用队列机制管理，支持并发执行，默认并发数为 2，

可通过 `FH_TASK_CONCURRENCY` 环境变量调整。每个任务拥有唯一的任务 ID（UUID 格式），

任务状态包括 `pending`（等待中）、`running`（执行中）、

`completed`（已完成）、`failed`（失败）、`cancelled`（已取消）。

创建任务：在任务管理页面点击新建任务按钮，弹出任务创建表单。表单包含任务目标（必填，

自然语言描述要完成的开发任务，支持多行输入）、执行模式（可选，普通对话模式/SWE Agent 模式/目标驱动模式）、

模型策略（可选，成本优先/能力优先/延迟优先）、指定模型（可选，

手动指定使用的模型)、上下文文件(可选,上传文件作为额外上下文注入)、任务标签(可选,为任务添加自定义标签)。填写完成后点击创建按钮,任务进入队列等待执行。

任务列表:任务管理页面以列表形式展示所有任务,每条任务记录显示任务 ID(前 8 位)、

任务目标摘要、状态标签(不同状态使用不同颜色标识)、创建时间、执行耗时、操作按钮。列表支持按状态筛选、按标签筛选、按时间范围筛选、关键词搜索。

列表支持分页和排序,用户可以按创建时间、耗时、状态等字段排序。执行中的任务会实时更新状态和进度。

任务详情:点击任务列表中的查看详情按钮或任务 ID,进入任务详情页面。

详情页面展示任务的完整信息,包括任务基本信息(任务 ID、目标、创建时间、开始时间、结束时间、状态、耗时、执行模式、模型策略、使用的模型、成本统计)、执行步骤时间线(以时间线形式展示代理执行的每一步,包括思考过程、工具调用名称、工具输入参数、工具输出结果、每步耗时,支持展开折叠查看详细内容)、文件变更列表(任务执行过程中创建、修改、删除的文件清单,点击可查看文件变更 diff)、

构建和测试结果、最终总结报告。任务输出支持 Markdown 渲染,代码块支持语法高亮,工具参数以结构化方式展示。

5.3 模型管理与审计日志

模型管理页面展示当前配置的所有模型提供商列表,包括模型 ID、类型(OpenAI 兼容/Ollama)、

基础地址、能力标签、每千 token 成本、状态(可用/不可用/已禁用)、

性能统计(总调用次数、成功次数、失败次数、成功率、平均延迟、累计成本)。

用户可以在该页面进行以下操作:添加新模型(点击添加模型按钮,填写模型配置信息,支持测试连接验证配置是否正确)、修改模型配置(点击模型条目进入编辑页面,修改 baseUrl、apiKey、tags、costPer1k 等字段)、

禁用启用模型（禁用后路由层不再选择该模型，但保留统计数据）、删除模型（从配置中移除模型提供商）、

查看性能详情（点击模型查看详细性能趋势图表，包括成功率趋势、延迟趋势、成本趋势、调用量趋势等）、手动切换当前模型（指定后续任务使用的模型，覆盖自动路由策略）。模型性能数据实时更新，每次模型调用后统计数据自动刷新。性能统计页面提供丰富的可视化图表，帮助用户了解各模型的实际表现，优化模型选择策略和成本控制。

审计日志页面记录系统中的所有操作事件，满足企业安全审计和合规要求。

每条审计记录包含时间戳（操作发生的精确时间）、操作用户（执行操作的用户标识，企业版显示用户名和角色）、操作类型（登录登出、任务创建取消删除、模型配置变更、系统配置修改、权限变更、数据导出等）、操作详情（操作的具体内容）、IP 地址（操作来源的 IP 地址）、用户代理（浏览器或客户端标识）。

审计日志支持按时间范围、操作类型、用户、IP 地址、关键词等多维度筛选，支持导出为 CSV 或 JSON 格式，支持长期归档存储。企业版审计日志功能更加强大，支持合规报告自动生成（按日周月生成审计合规报告，包含操作统计、异常行为分析、权限变更记录等）、异常行为检测（自动识别异常登录、批量数据导出、权限提升等可疑操作并告警）、操作溯源（通过审计日志追溯任何操作的完整链路）、日志防篡改（审计日志采用追加写入和哈希校验，防止日志被篡改或删除）。

审计日志是企业安全治理的重要组成部分，帮助企业满足等保 2.0、ISO 27001、SOC 2 等安全合规标准的要求。

第六章 高级功能

6.1 并行编排

并行编排在需要同时处理多个独立子任务的场景下使用。并行编排器将用户的大目标拆解为多个相互独立的子任务，

为每个子任务创建独立的 Git worktree 隔离工作区，并行启动多个子代理同时执行，最后汇总各工作区的执行结果。这种模式能够显著提升复杂任务的执行效率，特别适合涉及多个独立模块开发、多文件批量修改、多方案对比实验、多语言版本生成等场景。

并行编排的执行流程分为五个步骤：第一步目标拆解由规划器接收用户目标，分析任务的可并行性，将大目标拆解为多个相互独立的子任务，每个子任务有明确的目标描述、

预期产出和验收标准，规划器会确保子任务之间没有依赖关系；第二步工作区创建为每个子任务创建独立的 Git worktree，

worktree 基于主仓库的当前状态创建，各 worktree 之间完全隔离。

第三步并行执行同时启动多个子代理，每个子代理在其独立的 worktree 中执行分配的子任务。

执行受最大并发数限制（默认 3，可通过 `maxConcurrent` 参数配置），防止同时发起过多模型调用导致 API 限流或资源耗尽。当子任务数量超过最大并发数时，超出的子任务进入等待队列，待有子代理完成后自动启动。每个子代理独立执行思考-行动-观察循环，

调用工具完成子任务，直至任务完成或达到最大轮次。第四步结果汇总所有子任务完成后（包括成功和失败），

编排器汇总各 worktree 的执行结果，生成统一的总结报告，

报告包含每个子任务的执行状态、产出文件清单、执行耗时、错误信息等。

第五步工作区清理默认情况下任务完成后自动清理所有 worktree，

子代理的产物不自动合并到主仓库，用户可以设置 `cleanup=false` 保留 worktree，

手动检查和合并各子任务的产出。并行编排的优势在于执行效率高、隔离性好、

可追溯性强、资源可控。需要注意的是并行编排放适用于子任务之间相互独立的场景，

如果子任务之间存在依赖关系，则应使用串行执行模式或 SWE Agent 模式。

6.2 团队协作模式

团队协作模式模拟人类开发团队的协作方式，创建多个具有不同角色和职责的代理成员，

共享一个任务清单，各成员自主认领任务并执行，通过消息总线进行通信协作，最终汇总产出团队报告。这种模式适用于需要多种专业能力协作完成的复杂任务，例如前端开发加后端开发加测试的全栈项目，或架构设计加代码实现加代码评审的质量保障流程。

团队协作的核心组件包括三个：任务看板是共享的任务清单，支持添加任务、原子认领、完成标记、失败标记等操作，认领操作采用状态加所有者双校验保证原子性，当多个成员同时认领同一任务时只有一个成员能够成功认领，避免重复执行，任务看板支持任务优先级、标签、依赖关系等属性；消息总线是代理间的通信机制，支持点对点消息和广播消息，成员可以接收发给自己的消息和广播消息，实现协作沟通、状态同步、问题求助等交互，消息总线记录所有消息历史，支持追溯和分析团队协作过程；协调器管理团队的完整生命周期，包括创建成员、启动执行循环、监控执行进度、处理异常情况、汇总执行结果、生成团队报告。

团队成员配置：每个团队成员可以配置不同的角色描述，角色描述会注入到该成员代理的系统提示中，

引导其专注于特定领域的任务和采用相应的专业视角。例如可以配置前端工程师角色专注于 UI 设计、

交互开发、样式优化、前端性能等；后端工程师角色专注于 API 设计、数据库操作、业务逻辑、服务端性能等；测试工程师角色专注于测试用例编写、测试执行、Bug 发现、质量验证等；架构师角色专注于系统设计、技术选型、架构评审、技术债务管理等。成员从共享任务清单中认领任务时会优先选择与其角色匹配的任务，

但也可以认领其他类型的任务。团队模式支持配置每个成员的最大任务数，防止个别成员吞掉全部任务，保证负载均衡和角色分工。团队协作的执行流程：协调器首先根据配置创建团队成员，将用户目标拆解为任务列表添加到共享任务看板，然后启动所有成员的执行循环。每个成员独立循环执行从任务看板认领一个匹配的任务，通过消息总线广播认领通知，执行任务，完成后通过消息总线汇报结果，将任务标记为完成或失败，然后继续认领下一个任务直到任务看板为空或达到个人任务上限。

所有成员完成后协调器汇总所有任务的执行结果和消息总线记录生成团队报告。团队报告包括团队成员列表、任务统计、总体状态、每个任务的详细结果、消息总线历史。

6.3 MCP 集成

MCP (Model Context Protocol, 模型上下文协议) 是飞虹 Code 支持的开放协议, 允许代理连接外部 MCP 服务器, 获取额外的工具和上下文资源。MCP 是一个开放的标准协议,

定义了大语言模型与外部数据源和工具之间的通信规范, 使模型能够安全地访问外部服务和数据。

通过 MCP 集成, 飞虹 Code 可以连接各种第三方服务和数据源, 极大扩展代理的能力边界, 例如连接数据库执行 SQL 查询、连接项目管理系统读取和更新任务、

连接文档系统检索知识库、连接代码托管平台管理 Issue 和 PR、连接监控系统查询指标等。MCP 客户端实现了标准的 MCP 协议通信, 支持多种传输方式, 包括 stdio (标准输入输出, 适用于本地 MCP 服务器进程)、SSE (Server-Sent Events, 适用于远程 HTTP MCP 服务器)、HTTP (适用于 REST 风格的 MCP 服务器)。

MCP 客户端支持的核心操作包括工具列表查询 (获取 MCP 服务器提供的所有工具的描述和参数 schema)、

工具调用 (调用指定的工具并传入参数, 获取执行结果)、资源读取 (读取 MCP 服务器提供的资源内容,

如文件、数据库记录、API 响应等)、资源列表查询 (获取可用资源列表)、提示模板获取 (获取 MCP 服务器提供的提示模板)。MCP 客户端自动处理协议握手、消息序列化、错误处理、连接管理、自动重连等底层细节, 代理可以像调用内置工具一样透明地调用 MCP 工具。

MCP 配置通过环境变量或配置文件进行, 用户可以配置多个 MCP 服务器, 每个服务器指定名称、传输方式、连接参数。代理启动时自动连接所有配置的 MCP 服务器, 进行协议握手, 获取各服务器提供的工具列表, 并将这些工具注册到代理的可用工具集中。

MCP 集成的安全机制包括传输安全（支持 HTTPS/TLS 加密传输）、
认证安全（支持 API Key、Bearer Token、OAuth 2.0 等多种认证方式）、
工具白名单（用户可以配置允许调用的 MCP 工具白名单）、输入验证（MCP 客户端对工具调用参数进行 schema 验证）、
输出过滤（对工具返回的敏感信息进行脱敏处理）。

6.4 Webhook 集成

Webhook 功能允许飞虹 Code 接收外部系统的 HTTP 请求触发自动化任务（进站 Webhook），

也可以在任务完成时向外部系统推送通知（出站 Webhook）。通过 Webhook 集成，
飞虹 Code 可以与 CI/CD 流水线、项目管理系统、即时通讯工具、
监控告警系统等无缝集成，实现自动化 workflow。例如代码提交到 GitHub 后自动触发代码评审任务，

任务完成后将评审结果推送到企业微信群；监控系统检测到服务异常后自动触发故障排查任务，

代理分析日志并生成修复建议；项目管理系统创建新任务后自动触发开发任务，
代理完成后更新任务状态。进站 Webhook：飞虹 Code 提供标准的 Webhook 接收端点，
外部系统可以通过 HTTP POST 请求触发自动化任务。进站 Webhook 支持多种认证方式确保请求来源可信：

HMAC 签名校验（请求头携带 X-Signature 签名，使用共享密钥对请求体进行 HMAC-SHA256 计算，

服务端验证签名是否匹配）、企业微信回调签名校验（兼容企业微信回调的 msg_signature 签名验证机制）、

Bearer Token 认证、IP 白名单（仅允许白名单内的 IP 地址发起 Webhook 请求）。

进站 Webhook 的请求体包含任务触发信息，包括任务目标（要执行的开发任务描述）、
执行模式、模型策略、回调 URL（任务完成后推送结果的地址）、自定义元数据等。

服务端验证请求可信后创建任务并加入执行队列，立即返回任务 ID 和状态，

任务在后台异步执行。出站 Webhook：当任务状态变更（创建开始完成失败取消）或任务执行过程中产生重要事件时，

飞虹 Code 可以向配置的回调 URL 推送 Webhook 通知。

出站 Webhook 支持渠道白名单机制，用户可以配置 FH_CHANNEL_ALLOW 环境变量指定允许推送的 URL 白名单（逗号分隔的 URL 模式列表，

支持通配符），未在白名单中的 URL 不允许推送，防止数据外泄到未授权地址。

出站 Webhook 的推送内容包含事件类型、任务 ID、任务状态、

执行结果、错误信息、时间戳、签名（使用 HMAC-SHA256 对请求体签名，

接收方可以验证消息来源和完整性）等。接收方可以根据推送的事件类型和任务状态进行相应处理。

出站 Webhook 支持重试机制，当推送失败时自动重试，采用指数退避策略，

最多重试 3 次，确保通知可靠送达。Webhook 集成的典型应用场景包括 CI/CD 集成（代码提交后自动触发代码评审和测试任务，

结果回写到 PR 评论）、项目管理集成（项目管理系统创建新需求后自动触发开发任务，

代理完成后更新任务状态并关联代码变更）、即时通讯集成（在企业微信钉钉群中通过机器人命令触发开发任务，

任务完成后在群中推送结果通知）、监控告警集成（监控系统检测到服务异常后自动触发故障排查任务，

代理分析日志定位问题生成修复建议推送告警渠道）、自动化 workflow（通过 Webhook 串联多个系统和工具，

实现从需求提出到代码实现测试验证部署上线的全流程自动化）。

第七章 配置参考

7.1 环境变量详解

飞虹 Code 通过环境变量进行运行时配置，所有环境变量均以 FH_ 前缀开头，

便于识别和管理，避免与其他软件的环境变量冲突。环境变量可以通过多种方式设置：

系统环境变量（在操作系统层面永久设置，适用于全局默认配置）、.env 文件（放置在工作目录或用户主目录下，

软件启动时自动加载，适用于项目特定配置）、命令行参数（通过 -- 前缀的参数临时设置，

优先级最高，适用于单次运行的临时配置）。配置加载优先级为命令行参数大于 .env 文件大于系统环境变量大于默认值，

当多个来源同时配置同一选项时优先级高的来源覆盖优先级低的来源。核心配置环境变量包括：

FH_PROVIDERS（模型提供商 JSON 数组，是使用在线模式的必要配置）；

FH_MODEL_NAME（默认使用的模型名称，指定后路由层优先选择该名称的模型）；

FH_MODEL_STRATEGY（模型选择策略，可选 cost 成本优先默认、

capability 能力优先、latency 延迟优先)；FH_MAX_RETRIES (模型调用最大重试次数，默认 3)；FH_MAX_CYCLES (代理最大执行轮次，默认 50，防止代理无限循环)；FH_OFFLINE (强制离线模式，设置为 true 时即使配置了 FH_PROVIDERS 也使用 Mock 模型)；

FH_HOME_DIR (配置和数据存储目录，默认 ~/.feihong-code)。

Web 控制台配置环境变量包括：FH_WEB_PORT (Web 控制台监听端口，默认 8080)；FH_WEB_HOST (Web 控制台监听地址，

默认 0.0.0.0，允许外部访问，设置为 127.0.0.1 仅允许本地访问)；

FH_WEB_TOKEN (Web 控制台访问令牌，配置后访问控制台需要输入令牌验证身份，生产环境强烈建议配置)；FH_TASK_CONCURRENCY (Web 任务队列并发数，

默认 2，控制同时执行的任务数量，根据系统资源和 API 限流情况调整)；

FH_WEB_CORS_ORIGIN (CORS 允许的源，默认 *，

允许所有来源跨域访问，生产环境建议设置为具体的域名)。企业级配置环境变量包括：

FH_ENTERPRISE (是否启用企业级功能，设置为 true 启用多租户、

RBAC、审计日志、配额管理等企业级功能)；FH_TENANT_ID (当前租户 ID，

多租户模式下指定当前操作所属的租户)；FH_RBAC_ENABLE (是否启用 RBAC 权限控制，默认 true)；FH_AUDIT_ENABLE (是否启用操作审计日志，

默认 true)；FH_QUOTA_ENABLE (是否启用资源配额，

默认 true)；FH_LOG_LEVEL (日志级别，可选 debug 详细调试信息、

info 常规信息默认、warn 警告信息、error 仅错误信息)；

FH_LOG_FILE (日志文件路径，不设置则仅输出到控制台，设置后同时输出到文件)。

安全和沙箱配置环境变量包括：FH_SANDBOX_MODE (沙箱模式，

可选 full 完全模式允许所有操作默认、approval 审批模式危险操作需要人工审批、read-only 只读模式仅允许只读操作禁止文件写入和 Shell 执行)；

FH_CHANNEL_ALLOW（出站渠道白名单，逗号分隔的 URL 模式列表，

不设置则全放行，设置后仅允许向白名单内的 URL 推送 Webhook 通知）；

FH_MAX_OUTPUT_LINES（Shell 命令最大输出行数，

默认 200，超过部分自动截断，防止输出过多导致上下文溢出）；FH_MAX_FILE_SIZE（文件操作最大大小限制，

默认 10MB，超过大小的文件不允许读取或写入）；FH_ALLOWED_COMMANDS（允许执行的 Shell 命令白名单，

逗号分隔，不设置则允许所有命令，设置后仅允许执行白名单内的命令）。

7.2 配置文件详解

除环境变量外，飞虹 Code 还支持通过 JSON 配置文件进行更精细和结构化的配置。

配置文件默认路径为 `~/.feihong-code/config.json`，

也可以通过 `FH_CONFIG_FILE` 环境变量指定自定义路径。

配置文件采用 JSON 格式，支持 JSONC（带注释的 JSON），

配置项与环境变量一一对应，但配置文件支持更复杂的嵌套结构，例如模型提供商的详细配置、

技能启用列表、自定义评审规则、RBAC 角色定义等，这些复杂配置通过环境变量设置不够方便。

配置文件的主要字段和结构说明：`models`（模型配置对象，包含 `providers` 数组（模型提供商列表，

每个元素包含 `id`、`type`、`baseUrl`、`apiKey`、`tags`、

`costPer1k` 等字段，与 `FH_PROVIDERS` 环境变量格式一致）、

`defaultStrategy`（默认模型选择策略，`cost/capability/latency`）、

`budgetPerTaskUsd`（每个任务的最大预算，单位美元）、

`maxRetries`（最大重试次数））；`runtime`（运行时配置对象，

包含 `maxCycles`（最大执行轮次）、`offline`（是否离线模式）、

`logLevel`（日志级别）、`logFile`（日志文件路径）、`homeDir`（数据存储目录））；

`web`（Web 控制台配置对象，包含 `port`（端口）、`host`（监听地址）、

`token`（访问令牌）、`corsOrigin`（CORS 允许源）、

taskConcurrency（任务并发数））。

enterprise（企业级配置对象，包含 enable（是否启用企业级功能）、multiTenant（是否启用多租户）、rbac（是否启用 RBAC）、audit（是否启用审计日志）、quota（是否启用配额管理））；skills（技能配置对象，包含 enabled 数组（启用的技能名称列表，不设置则启用所有已安装技能）、disabled 数组（禁用的技能名称列表）、customDir（自定义技能目录路径）、marketUrl（技能市场 URL，不设置使用默认官方市场））；review（代码评审配置对象，包含 enabledRules（启用的规则 ID 列表，不设置则启用全部规则）、disabledRules（禁用的规则 ID 列表）、customRules（自定义规则数组，每个规则包含 id、description、severity、pattern 正则表达式或 astPattern AST 模式、message 问题描述、fix 修复建议等字段）、exclude（排除文件模式数组，glob 格式，匹配的文件不参与评审）、threshold（阻断阈值，critical/high/medium/low））；sandbox（沙箱配置对象，包含 mode（沙箱模式，full/approval/read-only）、allowedCommands（允许的命令白名单）、maxOutputLines（最大输出行数）、maxFileSize（最大文件大小））；logging（日志配置对象，包含 level（日志级别）、file（日志文件路径）、format（日志格式，json/text）、maxSize（日志文件最大大小，超过后自动轮转）、maxFiles（保留的日志文件数量））。配置文件的加载和热更新：软件启动时自动加载配置文件，解析配置项并应用到运行时。配置文件支持 include 指令，可以通过引用其他配置文件，便于组织大型配置和共享公共配置。配置变更后需要重启软件生效，

但部分运行时配置（如模型策略、日志级别、技能启用列表）支持通过 Web 控制台或 API 热更新，

无需重启即可生效。配置文件支持环境变量插值，在配置值中使用环境变量引用语法引用环境变量的值，

软件加载配置时自动替换为环境变量的实际值，避免在配置文件中明文存储敏感信息。

配置验证：软件加载配置文件时自动验证配置项的类型和取值范围，发现无效配置时输出警告并使用默认值，

确保软件能够正常启动。

第八章 常见问题与故障排查

8.1 安装与启动问题

问题一：`npm install -g feihong-code` 报权限错误（EACCES）。

原因分析：npm 全局安装目录需要管理员权限，当前用户没有写入权限。

解决方案：macOS/Linux 系统使用 `sudo npm install -g feihong-code` 命令以管理员权限安装；

Windows 系统右键点击命令提示符或 PowerShell 选择以管理员身份运行然后执行安装命令；

推荐方案是修改 npm 全局安装目录到用户主目录下，执行 `npm config set prefix ~/.npm-global` 设置新目录，

然后将 `~/.npm-global/bin` 添加到 PATH 环境变量中（在 `~/.bashrc` 或 `~/.zshrc` 中添加 `export PATH=$HOME/.npm-global/bin:$PATH`），

执行 `source ~/.bashrc` 重新加载配置后即可无需管理员权限进行全局安装。

问题二：执行 `fhcode` 命令提示 `command not found` 或不是内部或外部命令。

原因分析：npm 全局安装目录未添加到 PATH 环境变量，或安装未成功完成。

解决方案：首先确认安装是否成功执行 `npm list -g feihong-code` 查看是否已安装，

如果未安装重新执行安装命令；查看 npm 全局安装目录执行 `npm config get prefix` 获取目录路径，

确认该目录下的 `bin` 子目录中是否存在 `fhcode` 命令文件；

将 npm 全局安装目录的 `bin` 子目录添加到 PATH 环境变量中，

Windows 系统通过系统属性环境变量编辑 PATH，macOS/Linux 在 shell 配置文件中添加 `export PATH=npm-prefix/bin:$PATH`；

重启终端或执行 `source` 命令使环境变量生效，再次执行 `fhcode --version` 验证。

问题三：启动后提示未配置任何模型 provider 进入离线模式。

原因分析：未设置 FH_PROVIDERS 环境变量，或设置的格式不正确导致解析失败，

或配置的模型列表为空。解决方案：如果希望使用在线模式按照模型配置章节的格式正确设置 FH_PROVIDERS 环境变量，

注意 JSON 格式的引号和逗号，在 Windows PowerShell 中使用环境变量设置语法：

验证配置是否生效执行 `fhcode config` 命令查看当前配置，

确认 `models.providers` 列表非空且配置正确；确认 API 密钥有效且有足够额度，

`baseUrl` 地址正确可访问；如果只是想体验功能或进行测试可以忽略此提示使用离线模式，

离线模式功能完整代理能够执行真实的文件读写和 Shell 操作。

问题四：启动时报错 `Cannot find module` 或模块未找到。

原因分析：安装过程中依赖包下载不完整，或 Node.js 版本不兼容，

或全局安装目录损坏。解决方案：确认 Node.js 版本满足要求（18.0.0+）执行 `node --version` 查看版本，

版本过低请升级 Node.js；尝试重新安装执行 `npm uninstall -g feihong-code` 卸载后重新 `npm install -g feihong-code` 安装；

清理 npm 缓存后重新安装执行 `npm cache clean --force` 清理缓存；

如果是源码编译安装确认已执行 `npm install` 安装依赖然后重新执行 `npm run build` 编译。

8.2 模型调用问题

问题一：模型调用返回 401 Unauthorized（未授权）。

原因分析：API 密钥无效、已过期、被吊销，或 API 密钥与 `baseUrl` 不匹配，

或 `baseUrl` 地址错误。解决方案：检查 FH_PROVIDERS 中的 `apiKey` 是否正确注意复制时不要包含多余的空格或换行；

确认 API 密钥未过期且账户有足够额度登录模型服务商的控制台查看密钥状态和用量；

确认 `baseUrl` 地址与 API 密钥匹配 DeepSeek 的 `baseUrl` 是 `api.deepseek.com/v1` 通义千问的 `baseUrl` 是 `dashscope.aliyuncs.com/compatible-mode/v1` 不同服务商的 `baseUrl` 不同；

如果密钥已失效重新生成 API 密钥并更新配置。问题二：模型调用返回 429 Too Many Requests（请求过多）。

原因分析：API 调用频率超过服务商的限流阈值，或并发数过高导致短时间内发起大量请求，

或账户的 RPM（每分钟请求数）/TPM（每分钟 token 数）配额耗尽。

解决方案：降低 `FH_TASK_CONCURRENCY` 并发数减少同时执行的任务数量；

降低并行编排的 `maxConcurrent` 参数减少同时运行的子代理数量；

配置多个模型提供商路由层会自动轮换到未限流的模型分散请求压力；等待一段时间后重试路由层内置指数退避机制会自动处理瞬时限流；

如果频繁遇到限流考虑升级 API 套餐获得更高配额或使用本地 Ollama 模型避免云端限流。

问题三：模型调用超时或网络连接失败。原因分析：网络连接不稳定，或 `baseURL` 地址无法访问（防火墙拦截、

DNS 解析失败、服务端故障），或模型服务响应时间过长超过超时阈值。

解决方案：检查网络连接尝试 `ping` 或 `curl` 测试 `baseURL` 地址的连通性和延迟；

确认 `baseURL` 地址正确注意 URL 末尾是否需要 `/v1` 后缀（不同服务商要求不同）；

如果在需要代理的网络环境中配置 `HTTP_PROXY` 和 `HTTPS_PROXY` 环境变量指定代理服务器；

切换到其他可用模型（如 Ollama 本地模型）路由层会自动轮换可用模型并重试；

如果是模型服务端故障等待服务商恢复后重试。问题四：模型返回 `context length exceeded` 或输入超过最大长度。

原因分析：对话上下文过长包含的 `token` 数量超过模型的最大上下文窗口限制，

常见于长对话、大文件内容注入、多轮迭代后上下文累积等场景。解决方案：

使用 `/clear` 命令清空当前对话上下文开始新的对话；减少注入的上下文文件大小限制 32KB 避免注入过大的文件；

软件内置上下文压缩机制当上下文接近模型限制时自动触发压缩将历史对话摘要为紧凑形式如果压缩效果不佳可以手动 `/clear` 后重新描述需求；

选择支持更长上下文的模型（如带有 `long-context` 标签的模型）通过 `--strategy capability` 或 `--model` 参数指定长上下文模型；

将大任务拆分为多个小任务分别在独立会话中处理避免单会话上下文过长。

8.3 执行与验证问题

问题一：代理执行 Shell 命令被拒绝提示 `command not allowed` 或 `sandbox restriction`。

原因分析：安全沙箱限制当前沙箱模式不允许执行该命令，或命令匹配危险模式被拦截。

解决方案：检查 `FH_SANDBOX_MODE` 配置 `full` 模式允许所有命令 `approval` 模式需要人工审批 `read-only` 模式仅允许只读命令；

如果需要执行特定命令可以临时调整沙箱模式为 `full` 或在配置文件的 `sandbox.allowedCommands` 白名单中添加该命令；

确认命令不是危险操作如 `rm -rf` 根目录、`curl` 管道 `bash`、`sudo`、`eval`、`exec`、`nc`、`telnet` 等这些命令出于安全考虑默认被拦截；

对于需要执行的构建和测试命令（如 `npm run build`、`npm test`）通常是允许的如果被拦截检查白名单配置。

问题二：构建验证持续失败代理无法自动修复。原因分析：错误类型超出代理的修复能力（如环境配置问题、

依赖版本冲突、系统级问题），或修复轮次达到上限，或代理的修复方向不正确。

解决方案：查看任务详情中的错误信息和代理的修复尝试人工分析根本原因；

如果是环境配置问题（如缺少系统依赖、`Node.js` 版本不兼容、

端口被占用）先在本地手动解决环境问题再让代理处理代码层面的修复；

增加 `FH_MAX_CYCLES` 配置允许更多迭代轮次给代理更多修复机会；

对于复杂的构建问题建议将任务拆分为更小的子任务逐步解决；如果代理反复尝试同一错误方案使用 `/clear` 重置上下文后重新描述任务并提供更多错误上下文信息引导代理正确分析。

问题三：代理执行任务时修改了不应该修改的文件或删除了重要文件。原因分析：

代理的工具调用参数错误或任务目标描述不够明确导致代理误解操作范围或自我修复过程中误操作。

解决方案：立即使用 `git` 恢复被误修改的文件（`git checkout` 文件）或恢复被删除的文件（`git restore` 文件）如果文件未纳入版本控制检查是否有备份；

使用 `read-only` 沙箱模式（`FH_SANDBOX_MODE=read-only`）限制代理只能执行只读操作禁止文件写入和删除特别适用于代码评审和分析场景；

在任务目标中明确指定操作范围如仅修改 `src/api` 目录下的文件不要修改其他目录；

使用 `approval` 沙箱模式代理的文件写入和删除操作需要人工审批确认后才执行防止误操作；

定期提交代码到版本控制确保任何误操作都可以回滚。问题四：任务执行时间过长代理似乎陷入循环。

原因分析：代理在自我修复过程中反复尝试无效方案或工具调用返回结果不明确导致代理无法判断是否完成或最大轮次设置过高。

解决方案：查看任务详情中的执行步骤判断代理是否在重复相同的操作或反复犯同样的错误；

如果确认代理陷入循环手动取消任务（在 Web 控制台点击取消按钮或在终端按 `Ctrl+C`）；

分析循环原因调整任务描述或提供更多指导信息后重新执行；降低 `FH_MAX_CYCLES` 配置（如从 50 降到 20）防止代理无限循环；

如果是工具调用结果不明确检查工具是否正常工作输出是否符合预期格式必要时手动验证工具执行结果。

8.4 Web 控制台问题

问题一：Web 控制台无法访问浏览器提示无法访问此网站或连接超时。

原因分析：Web 服务未启动或端口被其他程序占用或防火墙阻止入站连接或监听地址配置不正确。

解决方案：确认 Web 服务已启动执行 `fhcode web` 命令启动服务观察终端输出确认服务正常启动并监听端口；

检查端口是否被占用 Windows 执行 `netstat` 查看端口占用 Linux/macOS 执行 `lsof` 查看端口占用如果端口被占用使用 `--port` 参数更换端口或停止占用端口的程序；

检查防火墙设置 Windows 在 Windows Defender 防火墙中添加入站规则允许该端口 Linux 使用 `ufw` 或 `firewall-cmd` 开放端口；

确认监听地址如果仅需本地访问设置 `FH_WEB_HOST=127.0.0.1` 如果需要外部访问保持默认 `0.0.0.0`；

确认访问地址正确格式为 `http://服务器IP:端口号` 如 `http://localhost:8080`。

问题二：访问 Web 控制台提示 `Invalid token` 或访问被拒绝。

原因分析：配置了 `FH_WEB_TOKEN` 但输入的令牌不正确或令牌配置后未重启服务或浏览器缓存了旧的无效令牌。

解决方案：确认输入的访问令牌与 `FH_WEB_TOKEN` 环境变量的值完全一致注意区分大小写不要包含多余空格；

如果修改了 `FH_WEB_TOKEN` 需要重启 Web 服务使新令牌生效；

清除浏览器缓存和 Cookie 或使用无痕隐私模式重新访问；如果忘记令牌可以在服务端查看 `FH_WEB_TOKEN` 环境变量的值或重启服务时设置新的令牌；

生产环境建议配置复杂的随机令牌并定期更换保障控制台安全。

问题三：Web 控制台任务列表不更新或任务执行状态卡在执行中。原因分析：

WebSocket 连接断开导致实时推送失效或任务执行进程异常退出但状态未更新或浏览器标签页长时间未活动导致连接挂起。

解决方案：刷新浏览器页面重新建立 WebSocket 连接任务状态会重新同步；

检查服务端日志确认任务进程是否正常运行如果进程异常退出任务状态会自动更新为 `failed`；

在任务详情页面查看任务的执行输出判断任务是否真的在执行还是已经卡住；

如果任务确实卡住手动取消任务后重新创建；检查网络连接稳定性 WebSocket 需要稳定的网络连接如果网络不稳定可能导致推送延迟或丢失。

问题四：Web 控制台界面显示异常样式错乱或按钮无法点击。原因分析：

浏览器缓存了旧版本的静态资源（CSS/JS）或浏览器版本过低不支持某些 CSS/JS 特性或静态资源加载失败（404 错误）。

解决方案：强制刷新浏览器（`Ctrl+F5` 或 `Cmd+Shift+R`）清除缓存重新加载静态资源；

使用现代浏览器访问推荐 Chrome 90+、Firefox 88+、Safari 14+、Edge 90+不支持 IE 浏览器；打开浏览器开发者工具（F12）查看 Console 和 Network 面板检查是否有 JavaScript 错误或资源加载失败；

如果静态资源 404 确认 Web 服务的静态资源目录配置正确 dist/web/public 目录存在且包含 index.html、css/style.css、js/app.js 等文件；如果是自定义部署确认静态资源已正确复制到服务目录。

第九章 详细教程与实战案例

9.1 教程一：从零创建一个 Node.js REST API 项目

本教程将演示如何使用飞虹 Code 从零开始创建一个完整的 Node.js REST API 项目，包括项目初始化、依赖安装、代码编写、测试运行等全流程。通过本教程，您将了解飞虹 Code 在实际开发场景中的使用方法和最佳实践。教程假设您已经完成飞虹 Code 的安装和模型配置，

能够正常使用在线模式或离线模式。如果尚未完成安装，请参考第二章安装部署章节的说明进行操作。

第一步：创建项目目录并启动飞虹 Code。在终端中执行 `mkdir my-api && cd my-api` 创建并进入项目目录，

然后执行 `fhcode` 命令启动交互式对话界面。在欢迎信息中确认当前模式为在线模式或离线模式，

配置的模型数量大于零。如果显示离线模式且您希望使用在线模式，请先配置 `FH_PROVIDERS` 环境变量后重启。

第二步：输入开发需求。在对话界面中输入以下需求描述：创建一个基于 Express.js 的 REST API 项目，

包含用户管理模块（CRUD 操作），使用内存存储，包含单元测试，项目结构清晰，代码注释完整。输入完成后按回车键发送，代理将开始执行任务。

第三步：观察代理执行过程。代理接收到目标后，首先进行任务规划，将大目标拆解为多个子任务：

初始化项目（package.json）、安装依赖（express、jest、supertest）、创建项目目录结构、编写服务器入口文件、

编写用户路由、编写用户控制器、编写内存数据存储、编写单元测试、运行测试验证。规划完成后，代理依次执行每个子任务，调用文件写入工具创建各个文件，调用 Shell 工具安装依赖和运行测试。

第四步：查看执行结果。任务完成后，代理输出总结报告，包括完成的子任务列表、生成的文件清单（`package.json`、`src/index.js`、`src/routes/users.js`、`src/controllers/userController.js`、`src/models/userStore.js`、`tests/users.test.js` 等）、测试运行结果（测试用例数量、通过数量、失败数量）、执行耗时统计。您可以在终端中查看生成的文件内容，执行 `npm test` 命令验证测试是否全部通过，执行 `npm start` 命令启动服务器并使用 `curl` 或 `Postman` 测试 API 接口。

第五步：迭代优化。如果对生成的代码有修改需求，可以继续在对中输入新的需求，例如：添加用户认证中间件，使用 `JWT` 进行身份验证；添加请求参数校验，使用 `zod` 库；添加 API 文档，使用 `Swagger`；将内存存储替换为 `SQLite` 数据库。代理会基于当前项目状态继续执行，支持上下文记忆，能够理解之前的代码结构和设计决策。

9.2 教程二：使用代码评审引擎发现并修复代码问题

本教程演示如何使用飞虹 Code 的代码评审引擎对现有代码库进行自动化审查，发现潜在问题并生成修复建议。代码评审引擎支持命令行、编辑器集成和 CI/CD 三种使用方式，

本教程以命令行方式为例。代码评审引擎基于预定义的规则集，扫描安全漏洞、性能瓶颈、代码规范、复杂度、可维护性、错误处理、资源泄漏等多个维度的问题，按严重程度分为 `critical`、`high`、`medium`、`low` 四个等级。

第一步：准备待评审的代码。在终端中进入待评审的项目目录，确保项目包含源代码文件（.js、

.ts、.py、.java 等）。代码评审引擎支持多种编程语言，规则集会根据文件类型自动适配。如果项目包含不需要评审的文件（如 node_modules、dist、build 等目录），可以通过 `--exclude` 参数排除。

第二步：执行代码评审。在终端中执行 `fhcode review .` 命令对当前目录进行全量评审。命令执行后，评审引擎扫描目录下的所有源代码文件，应用预定义规则集进行分析，生成结构化评审报告。默认输出人类可读的文本报告，包含每个问题的文件路径、行号、严重程度、问题描述、修复建议。如果需要 JSON 格式的结构化结果，可以添加 `--json` 参数。

第三步：分析评审报告。评审报告按严重程度排序，critical 级别的问题排在最前面。每个问题包含以下信息：严重程度标签（不同颜色标识）、文件路径和行号、规则 ID、问题描述（说明存在什么问题）、影响分析（说明问题可能导致的后果）、修复建议（给出具体的修复方案和代码示例）。您可以根据评审报告逐一修复问题，或使用 `--fix` 参数触发自动修复（实验性功能，仅支持部分规则的自动修复）。

第四步：设置质量门禁。在 CI/CD 流水线中，可以使用 `--threshold` 参数设置阻断阈值，当发现该级别及以上问题时命令返回非零退出码，阻断流水线执行。例如 `fhcode review . --threshold high` 表示当发现 high 或 critical 级别的问题时阻断构建。

这可以确保合并到主分支的代码满足最低质量标准。建议在项目初期设置较低的阈值（如 medium），

随着代码质量提升逐步提高阈值。

9.3 教程三：使用 Web 控制台管理团队开发任务

本教程演示如何使用飞虹 Code 的 Web 控制台进行团队开发任务管理，

包括创建任务、监控执行、查看结果、审计操作等。Web 控制台提供图形化界面，适合团队协作和任务集中管理场景。控制台支持多用户并发访问，任务队列机制确保任务有序执行，

审计日志记录所有操作满足合规要求。

第一步：启动 Web 控制台。在终端中执行 `fhcode web --port 8080 --token my-secret-token` 命令启动 Web 控制台服务。

`--port` 参数指定监听端口（默认 8080），`--token` 参数设置访问令牌（生产环境强烈建议配置）。

服务启动后，终端输出服务地址和访问令牌信息。在浏览器中输入 `http://服务器IP:8080` 访问控制台，

首次访问输入访问令牌后进入主界面。

第二步：创建开发任务。在任务管理页面点击新建任务按钮，弹出任务创建表单。

在任务目标字段输入详细的开发需求描述，例如：为用户模块添加密码重置功能，

包含重置链接生成、邮件发送、密码更新三个步骤，使用 JWT 生成有时效性的重置令牌。

选择执行模式（SWE Agent 模式适合复杂开发任务），选择模型策略（能力优先适合复杂任务），

可以上传上下文文件（如现有代码文件、设计文档等），添加任务标签（如 `feature`、`user-module` 等）。点击创建按钮，任务进入队列等待执行。

第三步：监控任务执行。任务创建后，在任务列表中可以看到任务状态从 `pending`（等待中）变为 `running`（执行中）。

点击任务 ID 进入详情页面，可以实时查看代理的执行过程，包括思考过程、

工具调用、工具结果、每步耗时。执行步骤以时间线形式展示，支持展开折叠查看详细内容。

文件变更列表实时更新，显示代理创建、修改、删除的文件。构建和测试结果实时显示，如果构建失败代理会自动触发自我修复。

第四步：查看任务结果。任务完成后，状态变为 `completed`（已完成）或 `failed`（失败）。

在详情页面查看最终总结报告，包括完成的子任务列表、生成或修改的文件清单、

构建和测试验证结果、耗时统计、成本统计。可以点击文件变更查看 diff，对比修改前后的代码差异。如果任务失败，可以查看错误信息和代理的修复尝试，分析失败原因后重新创建任务或手动修复。

第五步：审计操作记录。在审计日志页面查看所有操作记录，包括任务创建、任务取消、模型配置变更、系统配置修改等。每条记录包含时间戳、操作用户、操作类型、操作详情、IP 地址。支持按时间范围、操作类型、用户筛选，支持导出为 CSV 或 JSON 格式。企业版支持合规报告自动生成和异常行为检测，满足等保 2.0、ISO 27001 等安全合规标准要求。

9.4 教程四：配置多模型路由实现成本优化

本教程演示如何配置多模型路由，通过合理的模型选择策略实现成本优化和服务高可用。多模型路由是飞虹 Code 的核心能力，允许同时配置多个模型提供商，根据成本、能力、延迟等策略自动选择最优模型，并在模型故障时自动轮换。合理配置多模型路由可以显著降低 API 调用成本，提升服务稳定性。

第一步：评估模型需求。在配置多模型路由之前，需要评估项目的模型需求：

任务类型（代码生成、代码评审、文本理解、长文本处理等）、预算限制（每月 API 调用预算）、

延迟要求（交互式对话需要低延迟，批量任务可以容忍较高延迟）、上下文长度需求（长文件分析需要长上下文模型）。

根据需求选择合适的模型组合，通常包括一个低成本模型（用于简单任务）、一个高性能模型（用于复杂任务）、一个本地模型（用于敏感数据或离线场景）。

第二步：配置模型提供商。通过 `FH_PROVIDERS` 环境变量配置多个模型提供商。

每个提供商配置 `id`（唯一标识）、`type`（`openai-compatible` 或 `ollama`）、

baseUrl (API 地址)、apiKey (访问密钥)、tags (能力标签, 如 code-gen、cheap、long-context、local)、costPer1k (每千 token 成本, 用于成本计算和排序)。

建议为每个模型添加准确的能力标签和成本信息, 这是路由策略选择的依据。

例如低成本模型添加 cheap 标签, 长上下文模型添加 long-context 标签, 本地模型添加 local 标签。

第三步: 选择路由策略。根据项目需求选择合适的模型选择策略: cost (成本优先) 适合预算敏感场景,

路由层选择每千 token 成本最低的可用模型; capability (能力优先) 适合任务类型多样的场景,

路由层根据任务类型匹配具备相应能力标签的模型; latency (延迟优先) 适合交互式场景,

路由层选择历史平均响应时间最短的模型。可以通过 FH_MODEL_STRATEGY 环境变量设置默认策略,

或通过 --strategy 命令行参数临时切换, 或在 Web 控制台的模型管理页面手动指定当前任务使用的模型。

第四步: 监控模型性能。在 Web 控制台的模型管理页面查看各模型的实时性能统计, 包括总调用次数、成功次数、失败次数、成功率、平均延迟、累计成本。

性能数据以 JSONL 格式持久化存储, 支持历史趋势分析。根据性能数据调整模型配置:

如果某个模型频繁失败, 检查 API 密钥是否有效、baseUrl 是否正确、

账户是否有足够额度; 如果某个模型延迟过高, 考虑切换到其他模型或降低并发数;

如果成本超出预算, 调整路由策略为成本优先或降低并发数。

第五步: 配置故障自动轮换。多模型路由内置故障自动轮换机制, 当某个模型返回错误时自动切换到其他可用模型。

对于 400/401/403/404 等鉴权或模型类错误, 立即轮换不重试;

对于 429/500/502/503/504 等限流或服务端瞬时错误,

轮换后指数退避重试; 对于网络连接错误, 按可重试错误处理。建议至少配置两个以上模型提供商,

确保当一个模型故障时能够自动切换到另一个，保证服务连续性。

附录 A 命令速查表

基础命令：fhcode 启动交互式对话界面（默认命令）；fhcode chat 启动对话模式；

fhcode run goal 执行单次目标任务并输出结果；fhcode --version 或 -v 显示版本号；

fhcode --help 或 -h 显示帮助信息。开发命令：fhcode review 路径 对指定目录或文件进行结构化代码评审 --json 输出 JSON 格式 --threshold level 设置阻断阈值 --fix 自动修复（实验性）；

fhcode grill 对当前工作区进行深度拷问式审查；fhcode plan goal 仅执行任务规划输出方案不执行；

fhcode goal 启动目标驱动模式代理自主规划执行直至达成目标；

fhcode test 路径 生成并运行测试。系统命令：fhcode web 启动 Web 控制台 --port port 指定端口 --host host 指定地址 --token token 指定令牌；

fhcode eval 运行评估基准测试；fhcode verify 运行全量验证套件（M4-M9）；

fhcode model-stats 查看模型性能统计；fhcode config 查看当前运行配置；

fhcode doctor 环境诊断检查 Node.js 版本依赖完整性网络连通性等。

对话内斜杠命令：/plan 切换到规划模式；/grill 对当前代码进行拷问审查；

/review 路径 代码评审；/goal 启动目标驱动模式；/clear 清空对话上下文；

/exit 或 /quit 退出对话界面；/help 查看所有可用命令。

全局参数：--offline 强制离线模式；--strategy cost/capability/latency 模型选择策略；

--model name 指定模型名称；--max-cycles n 最大执行轮次；

--config path 配置文件路径；--log-level debug/info/warn/error 日志级别。

附录 B 配置速查表

模型配置：FH_PROVIDERS 模型提供商 JSON 数组（在线模式必要配置）；

FH_MODEL_NAME 默认模型名称；FH_MODEL_STRATEGY 模型策略（cost/capability/latency默认 cost）；

FH_MAX_RETRIES 最大重试次数（默认 3）；FH_OFFLINE 强制离线模式（true/false）。

Web 配置：FH_WEB_PORT 控制台端口（默认 8080）；

FH_WEB_HOST 监听地址（默认 0.0.0.0）；FH_WEB_TOKEN 访问令牌；

FH_TASK_CONCURRENCY 任务并发数（默认 2）；

FH_WEB_CORS_ORIGIN CORS 允许源（默认 *）。

企业配置：FH_ENTERPRISE 启用企业级功能（true/false）；

FH_TENANT_ID 租户 ID；FH_RBAC_ENABLE 启用 RBAC（默认 true）；

FH_AUDIT_ENABLE 启用审计日志（默认 true）；

FH_QUOTA_ENABLE 启用配额管理（默认 true）。

安全配置：FH_SANDBOX_MODE 沙箱模式（full/approval/read-only默认 full）；

FH_CHANNEL_ALLOW 出站渠道白名单（逗号分隔 URL）；

FH_MAX_OUTPUT_LINES Shell 最大输出行数（默认 200）；

FH_MAX_FILE_SIZE 文件最大大小（默认 10MB）；

FH_ALLOWED_COMMANDS 允许命令白名单。运行配置：

FH_MAX_CYCLES 最大执行轮次（默认 50）；FH_HOME_DIR 数据存储目录（默认 ~/.feihong-code）；

FH_CONFIG_FILE 配置文件路径；FH_LOG_LEVEL 日志级别（debug/info/warn/error默认 info）；

FH_LOG_FILE 日志文件路径。

附录 C 术语表

Agent（代理）：能够自主感知环境、做出决策、执行动作以达成目标的软件实体，

飞虹 Code 的核心执行单元。ReAct（Reasoning + Acting）：

一种代理执行模式，代理交替进行推理思考和行动执行，通过观察行动结果调整后续决策。

SWE Agent（Software Engineering Agent）：

软件工程代理，专门用于处理软件开发任务的高级代理模式，具备规划、执行、验证、修复完整闭环。MCP (Model Context Protocol)：模型上下文协议，大语言模型与外部数据源和工具之间的开放通信标准协议。

RBAC (Role-Based Access Control)：

基于角色的访问控制，通过为用户分配角色来管理权限的安全机制。Worktree (工作树)：Git 的工作树机制，允许在同一仓库的不同目录中同时检出不同分支，实现工作区隔离。R8: Android 的代码混淆和优化工具，通过重命名、移除无用代码、优化字节码等方式减小 APK 体积并增加逆向难度。

HMAC (Hash-based Message Authentication Code)：

基于哈希的消息认证码，一种使用密钥对消息进行签名和验证的安全机制。

CORS (Cross-Origin Resource Sharing)：

跨域资源共享，一种允许浏览器向不同源的服务器发起请求的安全机制。

WebSocket：一种在单个 TCP 连接上进行全双工通信的协议，

适用于需要实时双向数据传输的场景。JSONL (JSON Lines)：

每行一个 JSON 对象的文本格式，适合流式读写和追加写入，常用于日志和数据存储。

CI/CD (Continuous Integration/Continuous Deployment)：

持续集成/持续部署，自动化构建、测试和部署的软件开发实践。Token (令牌)：

在大语言模型中文本被切分为最小处理单元，模型按 token 数量计费 and 限制上下文长度。

Self-Heal (自我修复)：代理在遇到错误时自动分析原因并迭代修复直至问题解决的能力。

Context Compaction (上下文压缩)：当对话上下文过长时将历史消息摘要为紧凑形式以减少 token 占用的技术。

CLI (Command Line Interface)：命令行界面，

通过文本命令与软件交互的用户界面。API (Application Programming Interface)：

应用程序编程接口，软件组件之间交互的规范和协议。

附录 D 更新日志

V1.0.0（2026年8月23日）：首次正式发布。核心功能包括多模型路由（支持 DeepSeek/通义千问/Ollama/任意 OpenAI 兼容接口，

成本/能力/延迟三种选择策略，自动轮换与退避重试）、SWE Agent 全自动软件工程代理（规划-执行-验证-修复完整闭环，

自我修复机制）、代码评审引擎（多维度规则，四级严重程度，命令行/编辑器/CI-CD 三种使用方式）、

技能系统（可扩展技能框架，内置基础技能，技能市场，自定义技能）、

自进化与经验学习（经验积累、检索与应用，自进化闭环）、Web 控制台（任务管理、

模型管理、审计日志、实时推送、响应式设计）、CLI 交互界面（REPL 模式、

斜杠命令、多轮对话、上下文压缩）、离线模式（Mock 模型驱动，

无需 API 密钥，完整功能体验）、并行编排（多子任务并行执行，

Git worktree 隔离，最大并发控制）、团队协作模式（多角色代理，

共享任务看板，消息总线，团队报告）、MCP 集成（标准协议通信，

多传输方式，工具/资源/提示支持，安全机制）、Webhook 集成（进站触发，

出站通知，HMAC/企业微信签名认证，渠道白名单）、企业级功能（多租户隔离，

RBAC 权限控制，操作审计日志，资源配额管理）、Docker 容器化部署、

npm 全局安装、Windows/macOS/Linux 跨平台支持。

技术栈：TypeScript 5.6 + Node.js 18+；

Express.js Web 框架；Zod 4.x 数据验证；原生 HTML/CSS/JavaScript 前端（无框架依赖轻量高效）；

支持 npm 全局安装和 Docker 容器化部署；内置 OpenAI 兼容和 Ollama 模型适配器；

支持任意 OpenAI 兼容 API 接入；Git worktree 隔离；

R8 混淆加固（Android 端）。开源协议：MIT License。

源代码托管于 GitHub，npm 包名 feihong-code。

项目遵循语义化版本（Semantic Versioning）规范，

后续版本将持续增强代理能力、扩展工具生态、优化用户体验、完善企业级功能、

提升稳定性和性能。欢迎社区贡献代码、提交 Issue、分享技能和使用经验。

